70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A5

Taqsimlangan sun'iy intellekt tizimlarini ishlab chiqish

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A5
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A5
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada talaba taqsimlangan o'qitish va parallel inferensning asosiy mexanizmlarini CPU sandbox'da bajaradi: torch.distributed (gloo backend) bilan 2 jarayonli DDP, kuchli/kuchsiz masshtablash o'lchovi, halqa all-reduce algoritmini qo'lda yozish, gradient to'plash ekvivalentligi, multiprocessing Pool bilan parallel inferens, Amdal/Gustafson qonunlari va GPU klasteri uchun aloqa/hisob kalkulyatori. Real labdan farqi: GPU va NCCL yo'q — 2 ta CPU yadro «2 ta GPU» rolini o'ynaydi, modellar kichik MLP, ma'lumot sintetik yoki sklearn digits; mantiq (process group, DistributedSampler, DDP o'rami, all-reduce) esa haqiqiy klasterdagi bilan bir xil.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A5`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A5 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. DDP: gloo backend bilan 2 jarayonli taqsimlangan o'qitish
2. 2-qadam. Masshtablash: 1 vs 2 jarayon, S(N) va E(N)
3. 3-qadam. Halqa all-reduce ni NumPy'da qo'lda yozish va aloqa hajmi
4. 4-qadam. Gradient to'plash va ma'lumotlar parallelligi ekvivalentligi
5. 5-qadam. Parallel inferens: sharding, ishchilar puli, natijalarni yig'ish
6. 6-qadam. Amdal va Gustafson qonunlari grafigi
7. 7-qadam. Aloqa/hisob nisbati kalkulyatori (GPU klasteri uchun baho)
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. DDP: gloo backend bilan 2 jarayonli taqsimlangan o'qitish

MAQSAD

torch.multiprocessing orqali 2 ta jarayon (rank 0 va 1) ishga tushiriladi, har biri jarayonlar guruhiga qo'shiladi, DistributedSampler ma'lumotni kesishmaydigan ikki qismga bo'ladi, DDP o'rami gradientlarni har qadamda all-reduce qiladi. Oxirida parametrlar barcha ranklarda bir xil ekani (max farq 0) va jami ko'rilgan namunalar soni tekshiriladi. Talaba `EPOCHS`, `BS`, LR masshtablash qoidasini (`0.05 * world`) va `set_epoch` chaqiruvini o'zgartirib ko'rishi mumkin.

> **Kutilgan natija:** Ikkala rank o'z 4096 namunasini ko'radi, jami 8192; aniqlik ~0.95; ranklar orasidagi parametr farqi 0.00e+00 (DDP sinxron); o'tkazuvchanlik bir necha o'n ming namuna/s.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. DDP: 2 jarayonli taqsimlangan o'qitish (gloo backend, CPU)
# GPU va NCCL yo'q — shuning uchun CPU uchun mo'ljallangan gloo backend ishlatiladi.
# Mantiq (process group, DistributedSampler, DDP o'rami, all-reduce) haqiqiy GPU klasteridagi
bilan bir xil.
import os, time, random
import torch, torch.nn as nn, torch.distributed as dist
import torch.multiprocessing as mp
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data.distributed import DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP

N_SAMPLES, N_FEAT, N_CLS = 4096, 64, 8
EPOCHS, BS = 2, 64          # BS — har bir jarayon uchun paket (kuchsiz masshtablash rejimi)

def make_data():
    """Sintetik tasniflash ma'lumoti — barcha ranklarda bir xil urug' bilan yaratiladi."""
    g = torch.Generator().manual_seed(0)
    centers = torch.randn(N_CLS, N_FEAT, generator=g) * 0.6
    y = torch.randint(0, N_CLS, (N_SAMPLES,), generator=g)
    X = centers[y] + torch.randn(N_SAMPLES, N_FEAT, generator=g) * 1.5
    return TensorDataset(X, y)

def make_model():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(N_FEAT, 256), nn.ReLU(), nn.Linear(256, 256), nn.ReLU(),
nn.Linear(256, N_CLS))

def worker(rank, world, port):
    """Har bir jarayon shu funksiyani bajaradi — torchrun'dagi bitta 'rank' ga teng."""
    os.environ["MASTER_ADDR"] = "127.0.0.1"
    os.environ["MASTER_PORT"] = str(port)
    os.environ["GLOO_SOCKET_IFNAME"] = "lo"      # tarmoq yo'q — faqat loopback interfeysi
    torch.set_num_threads(1)                      # har jarayonga 1 yadro (2 yadro = 2 jarayon)
    dist.init_process_group("gloo", rank=rank, world_size=world)

    ds = make_data()
    sampler = DistributedSampler(ds, num_replicas=world, rank=rank, shuffle=True, seed=0)
    dl = DataLoader(ds, batch_size=BS, sampler=sampler, drop_last=True)
    model = DDP(make_model())                     # 0-rank parametrlari hammaga broadcast
qilinadi
```

```
    opt = torch.optim.SGD(model.parameters(), lr=0.05 * world, momentum=0.9)   # chiziqli LR
qoidasi
    lossf = nn.CrossEntropyLoss()

    t0 = time.time(); seen = 0
    for ep in range(EPOCHS):
        sampler.set_epoch(ep)                    # har epoxada boshqa aralashtirish
        for X, y in dl:
            opt.zero_grad(set_to_none=True)
            loss = lossf(model(X), y)
            loss.backward()                      # gradientlar chelaklarda avtomatik all-
reduce qilinadi
            opt.step(); seen += X.size(0)
    dt = time.time() - t0

    # Global ko'rsatkichlarni yig'amiz: namunalar soni va oxirgi yo'qotish
    stats = torch.tensor([seen, loss.item()], dtype=torch.float64)
    dist.all_reduce(stats, op=dist.ReduceOp.SUM)
    # Parametrlar barcha ranklarda bir xilmi? (DDP to'g'ri ishlaganining isboti)
    flat = torch.cat([p.detach().flatten() for p in model.parameters()])
    mx = flat.clone(); mn = flat.clone()
    dist.all_reduce(mx, op=dist.ReduceOp.MAX); dist.all_reduce(mn, op=dist.ReduceOp.MIN)
    diff = (mx - mn).abs().max().item()
    # Aniqlik — to'liq to'plamda (har rank hisoblaydi, 0-rank chiqaradi)
    with torch.no_grad():
        acc = (model(ds.tensors[0]).argmax(1) == ds.tensors[1]).float().mean().item()
    print(f"rank={rank} world={world}  namunalar(shu rank)={seen}  vaqt={dt:.2f}s", flush=True)
    if rank == 0:
        print("=== DDP natijasi (gloo, 2 jarayon) ===")
        print(f"jami ko'rilgan namunalar : {int(stats[0].item())}  ({EPOCHS} epoxa x
{N_SAMPLES})")
        print(f"o'rtacha oxirgi yo'qotish: {stats[1].item() / world:.4f}")
        print(f"aniqlik (to'liq to'plam) : {acc:.3f}")
        print(f"ranklar orasidagi parametr farqi: {diff:.2e}  -> {'BIR XIL (DDP sinxron)' if
diff < 1e-6 else 'FARQ BOR!'}")
        print(f"o'tkazuvchanlik: {stats[0].item() / dt:.0f} namuna/s")
    dist.destroy_process_group()

if __name__ == "__main__":
    port = random.randint(20000, 60000)          # port band bo'lmasligi uchun tasodifiy
    world = 2
    print(f"Ishga tushirish: world_size={world}, backend=gloo, MASTER_PORT={port}")
    mp.start_processes(worker, args=(world, port), nprocs=world, start_method="fork",
join=True)
    print("Izoh: GPU klasterida backend='nccl', ishga tushirish torchrun bilan bo'ladi; kod
mantig'i o'zgarmaydi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== DDP natijasi (gloo, 2 jarayon) ===`, `BIR XIL (DDP sinxron)`

TAJRIBA QILIB KO‘RING

- `sampler.set_epoch(ep)` satrini izohga olib, ikki epoxada bir xil aralashtirish bo'lishini tekshiring (yo'qotish qanday o'zgaradi?).
- `DDP(make_model())` o'rniga oddiy `make_model()` qo'ying — parametr farqi endi nolga teng bo'ladimi? Nega?
- `lr=0.05 * world` ni `lr=0.05` qilib, yakuniy yo'qotishni solishtiring (chiziqli LR qoidasi).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Masshtablash: 1 vs 2 jarayon, S(N) va E(N)

MAQSAD

Bir xil MLP uchun to'rt konfiguratsiya o'lchanadi: 1 jarayon (1 yadro) — baza T(1); 2 jarayonli DDP kuchli rejimda (global paket o'zgarmaydi, har rankka yarmi); kuchsiz rejimda (har rankda to'liq paket, global 2 baravar); va taqqoslash uchun 1 jarayon 2 yadro bilan (intra-op parallellik). Tezlashish S(N) va samaradorlik E(N) jadvalda chiqadi. Talaba `HID`, `GB`, `STEPS` ni o'zgartirib, model kattalashganda samaradorlik qanday o'zgarishini ko'radi.

> **Kutilgan natija:** Baza ~4-5 s; DDP kuchli rejimda S(2)≈1.6-1.8 (E≈80-90%), kuchsiz rejimda E≈90%; intra-op 2 yadro ham ~1.9x. Qadam 15-25 s davom etadi.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Masshtablash: 1 jarayon vs 2 jarayon (kuchli va kuchsiz masshtablash), S(N) va E(N)
# CPU sandbox: 2 yadro. Har jarayon 1 yadro oladi — bu "2 ta GPU" ning CPU muqobili.
import os, time, random
import torch, torch.nn as nn, torch.distributed as dist
import torch.multiprocessing as mp
from torch.nn.parallel import DistributedDataParallel as DDP

N_FEAT, HID, N_CLS = 64, 1024, 8
STEPS = 60                      # o'lchanadigan optimizatsiya qadamlari soni

def make_model():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(N_FEAT, HID), nn.ReLU(), nn.Linear(HID, HID), nn.ReLU(),
nn.Linear(HID, N_CLS))

def train_steps(model, bs, steps):
    """bs hajmli paketda `steps` ta qadam; vaqtni qaytaradi (2 ta isitish qadami hisobga
olinmaydi)."""
    opt = torch.optim.SGD(model.parameters(), lr=0.01)
    lossf = nn.CrossEntropyLoss()
    g = torch.Generator().manual_seed(1)
    X = torch.randn(bs, N_FEAT, generator=g); y = torch.randint(0, N_CLS, (bs,), generator=g)
    for i in range(steps + 2):
        if i == 2: t0 = time.time()
        opt.zero_grad(set_to_none=True)
        lossf(model(X), y).backward(); opt.step()
    return time.time() - t0

def worker(rank, world, port, bs_per_rank, q):
    os.environ.update(MASTER_ADDR="127.0.0.1", MASTER_PORT=str(port), GLOO_SOCKET_IFNAME="lo")
    torch.set_num_threads(1)
    dist.init_process_group("gloo", rank=rank, world_size=world)
    model = DDP(make_model())
    dt = train_steps(model, bs_per_rank, STEPS)
    if rank == 0: q.put(dt)
    dist.destroy_process_group()

def run_ddp(world, bs_per_rank):
    q = mp.get_context("fork").Queue()
    port = random.randint(20000, 60000)
    mp.start_processes(worker, args=(world, port, bs_per_rank, q), nprocs=world,
start_method="fork", join=True)
```

```
    return q.get()

if __name__ == "__main__":
    torch.set_num_threads(1)
    GB = 256                                          # global paket (kuchli masshtablash
uchun)
    t1 = train_steps(make_model(), GB, STEPS)         # T(1): bitta jarayon, 1 yadro
    t2_strong = run_ddp(2, GB // 2)                   # kuchli: global paket o'zgarmaydi -
> har rankka 128
    t2_weak = run_ddp(2, GB)                          # kuchsiz: har rankda 256 -> global
512
    torch.set_num_threads(2)
    t1_2thr = train_steps(make_model(), GB, STEPS)    # taqqoslash: 1 jarayon, lekin 2
yadro (intra-op)

    rows = [("1 jarayon (1 yadro) — baza", GB, t1, 1),
            ("2 jarayon DDP, kuchli (global 256)", GB, t2_strong, 2),
            ("2 jarayon DDP, kuchsiz (global 512)", 2 * GB, t2_weak, 2),
            ("1 jarayon, 2 yadro (intra-op)", GB, t1_2thr, 2)]
    print("=== Masshtablash jadvali (gloo, CPU) ===")
    print(f"{'Konfiguratsiya':38s} {'global paket':>12s} {'vaqt,s':>8s} {'namuna/s':>10s}
{'S(N)':>6s} {'E(N)':>7s}")
    for name, gb, t, n in rows:
        thr = gb * STEPS / t
        # kuchsiz rejimda tezlashish o'tkazuvchanlik nisbati bilan hisoblanadi
        s = (thr / (GB * STEPS / t1))
        print(f"{name:38s} {gb:12d} {t:8.2f} {thr:10.0f} {s:6.2f} {100 * s / n:6.0f}%")
    print("\nIzoh: S(N)=T(1)/T(N) (kuchsiz rejimda namuna/s nisbati), E(N)=S(N)/N.")
    print("Kuchli masshtablashda har rankka kamroq ish tegadi — samaradorlik odatda pastroq.")
    print("Haqiqiy GPU klasterida bir tugunda E(2) ~ 90-95%, tugunlar orasida 70-90%
kutiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Masshtablash jadvali (gloo, CPU) ===`

TAJRIBA QILIB KO‘RING

- `HID` ni 1024 dan 256 ga tushiring: kichik modelda aloqa ulushi oshib, E(2) pasayishini kuzating.
- `GB` ni 64 qiling — kuchli masshtablashda har rankka 32 namuna tegadi; samaradorlik nima bo'ladi?
- DDP o'rnida `bucket_cap_mb=1` parametri bilan sinab ko'ring (ko'proq, kichikroq all-reduce).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Halqa all-reduce ni NumPy'da qo'lda yozish va aloqa hajmi

MAQSAD

N ta ishchining gradient vektorlari uchun halqa all-reduce (N-1 reduce-scatter + N-1 all-gather qadami) simulyatsiya qilinadi, natija oddiy yig'indi bilan solishtiriladi va har ishchi yuborgan baytlar `2(N-1)/N * S` formulasiga mos kelishi ko'rsatiladi. So'ng ResNet-50 gradientlari (102 MB) uchun NVLink, InfiniBand va 10 GbE tarmoqlarda aloqa vaqti hisoblanadi.

> **Kutilgan natija:** N=2,4,8 uchun «natija to'g'ri=True» va yuborilgan baytlar formulaga aynan teng; 10 GbE da 8 GPU uchun ~143 ms, NVLink'da 0.3 ms.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Halqa (ring) all-reduce ni NumPy'da qo'lda amalga oshirish va aloqa hajmini
hisoblash
import numpy as np

def ring_allreduce(grads):
    """grads: ro'yxat, har bir ishchining gradient vektori (bir xil uzunlik).
    Qaytaradi: har ishchidagi natija (yig'indi) va har ishchi yuborgan baytlar soni."""
    N = len(grads); L = len(grads[0])
    chunks = np.array_split(np.arange(L), N)           # vektor N bo'lakka bo'linadi
    buf = [g.astype(np.float32).copy() for g in grads]  # har ishchining mahalliy buferi
    sent = [0] * N
    # 1-bosqich: reduce-scatter — N-1 qadam, har qadamda ishchi i bo'lak (i-k) ni o'ng
qo'shnisiga yuboradi
    for k in range(N - 1):
        pkts = []
        for i in range(N):
            c = chunks[(i - k) % N]
            pkts.append((c, buf[i][c].copy())); sent[i] += len(c) * 4
        for i in range(N):
            c, data = pkts[(i - 1) % N]                # chap qo'shnidan kelgan bo'lak
            buf[i][c] += data                          # qo'shib qo'yiladi (reduce)
    # 2-bosqich: all-gather — N-1 qadam, tayyor bo'lgan bo'lak halqa bo'ylab tarqatiladi
    for k in range(N - 1):
        pkts = []
        for i in range(N):
            c = chunks[(i + 1 - k) % N]
            pkts.append((c, buf[i][c].copy())); sent[i] += len(c) * 4
        for i in range(N):
            c, data = pkts[(i - 1) % N]
            buf[i][c] = data                           # almashtiriladi (gather)
    return buf, sent

print("=== Ring all-reduce tekshiruvi ===")
rng = np.random.default_rng(0)
for N in (2, 4, 8):
    L = 1000
    grads = [rng.standard_normal(L) for _ in range(N)]
    out, sent = ring_allreduce(grads)
    ref = np.sum(grads, axis=0)
    ok = all(np.allclose(o, ref, atol=1e-4) for o in out)
    S = L * 4
```

```
    theory = 2 * (N - 1) / N * S
    print(f"N={N:2d}: natija to'g'ri={ok}  har ishchi yubordi={sent[0]} bayt  formula 2(N-
1)/N*S={theory:.0f} bayt  "
          f"(sodda 'hammaga yuborish' bo'lsa {(N-1)*S} bayt)")

print("\n=== Aloqa hajmi va vaqti: ResNet-50 (25.6 mln parametr, FP32 = 102 MB) ===")
S_bytes = 25.6e6 * 4
nets = {"NVLink (600 GB/s)": 600e9, "InfiniBand 100 Gbit/s": 100e9 / 8, "Ethernet 10 Gbit/s":
10e9 / 8}
print(f"{'N GPU':>6s} {'bayt/GPU, MB':>13s} " + " ".join(f"{k:>24s}" for k in nets))
for N in (2, 4, 8, 16, 64):
    per = 2 * (N - 1) / N * S_bytes
    print(f"{N:6d} {per / 1e6:13.1f} " + " ".join(f"{per / bw * 1e3:22.1f}ms" for bw in
nets.values()))
print("\nXulosa: N oshganda har GPU yuboradigan hajm 2S ga intiladi (ishchilar sonidan deyarli
bog'liq emas),")
print("lekin tarmoq sekin bo'lsa (10 GbE) qadam sayin ~160 ms aloqa — hisoblash vaqtiga teng
yoki undan ko'p.")
print("BF16 gradient yuborilsa hajm 2 baravar kamayadi; gradient to'plash all-reduce
chastotasini kamaytiradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Ring all-reduce tekshiruvi ===`, `natija to'g'ri=True`

TAJRIBA QILIB KO‘RING

- `ring_allreduce` ni o'rtacha (SUM/N) qaytaradigan qilib o'zgartiring — DDP aynan shuni qiladi.
- Parametr-server sxemasini yozing (hamma 0-ishchiga yuboradi, u qaytaradi) va 0-ishchi yuboradigan hajmni N bo'yicha solishtiring.
- S_bytes ni BF16 (2 bayt) uchun qayta hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Gradient to'plash va ma'lumotlar parallelligi ekvivalentligi

MAQSAD

Bir xil 128 namunali global paket uchun uch usul taqqoslanadi: bitta katta paket; 4 ta mikro-paketda gradient to'plash (yo'qotish 1/K ga ko'paytiriladi); 2 «ishchi»da 64 tadan hisoblab gradientlarni o'rtachalash (DDP all-reduce'ning qo'lda nusxasi). Farqlar 1e-7 dan kichik chiqishi, 1/K ga bo'lmaslik esa gradientni K baravar kattalashtirishi ko'rsatiladi.

> **Kutilgan natija:** max|gA-gB| va max|gA-gC| ~1e-8; xato variantda farq ~0.3 va norma nisbati 4.00.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Gradient to'plash (accumulation) va ma'lumotlar parallelligi ekvivalentligi
# Uch usul bir xil 128 namunali global paket uchun bir xil gradient berishi kerak:
#  (A) bitta katta paket 128;  (B) 4 ta mikro-paket 32 — gradient to'planadi;
#  (C) "2 ishchi" har biri 64 — gradientlar o'rtachalanadi (DDP all-reduce ning qo'lda
nusxasi).
import torch, torch.nn as nn
torch.manual_seed(0)
N_FEAT, N_CLS, GB = 32, 5, 128
X = torch.randn(GB, N_FEAT); y = torch.randint(0, N_CLS, (GB,))
lossf = nn.CrossEntropyLoss()     # 'mean' — o'rtacha yo'qotish, shuning uchun bo'laklar vazni
muhim

def fresh():
    torch.manual_seed(1)
    return nn.Sequential(nn.Linear(N_FEAT, 64), nn.ReLU(), nn.Linear(64, N_CLS))

def flat_grad(m):
    return torch.cat([p.grad.flatten() for p in m.parameters()])

# (A) to'liq paket
mA = fresh(); lossf(mA(X), y).backward(); gA = flat_grad(mA)

# (B) gradient to'plash: 4 x 32, har mikro-paket yo'qotishi 1/4 ga ko'paytiriladi
mB = fresh(); K = 4
for i in range(K):
    xb, yb = X[i*32:(i+1)*32], y[i*32:(i+1)*32]
    (lossf(mB(xb), yb) / K).backward()        # .grad ichida yig'iladi — zero_grad
chaqirilmaydi
gB = flat_grad(mB)

# (C) ma'lumotlar parallelligi: 2 ishchi, har biri 64 namuna, so'ng all-reduce (o'rtacha)
workers = [fresh(), fresh()]                   # DDP broadcast — bir xil boshlang'ich
og'irliklar
grads = []
for r, m in enumerate(workers):
    lossf(m(X[r*64:(r+1)*64]), y[r*64:(r+1)*64]).backward()
    grads.append(flat_grad(m))
gC = torch.stack(grads).mean(0)                # all-reduce(SUM)/world_size

# (B') xato variant: 1/K ga bo'lmasdan to'plash — gradient K baravar katta chiqadi
mE = fresh()
for i in range(K):
    lossf(mE(X[i*32:(i+1)*32]), y[i*32:(i+1)*32]).backward()
gE = flat_grad(mE)
```

```
print("=== Gradient ekvivalentligi ===")
print(f"||gA|| (to'liq paket 128)            = {gA.norm():.6f}")
print(f"max|gA - gB| (to'plash 4x32)         = {(gA - gB).abs().max():.2e}")
print(f"max|gA - gC| (2 ishchi x 64, o'rtacha)= {(gA - gC).abs().max():.2e}")
print(f"max|gA - gE| (to'plash, 1/K siz!)    = {(gA - gE).abs().max():.2e}   <- xato:
||gE||/||gA|| = {gE.norm() / gA.norm():.2f}")
print("\nXulosa: 1 x 128 == 4 x 32 (to'plash) == 2 ishchi x 64 (DDP). Shuning uchun")
print("GPU yetmasa gradient to'plash bilan katta global paketni taqlid qilish mumkin, aloqa
chastotasi esa kamayadi.")
print("Eslatma: BatchNorm bo'lsa ekvivalentlik buziladi (statistika mikro-paketga bog'liq).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Gradient ekvivalentligi ===`

TAJRIBA QILIB KO‘RING

- Modelga `nn.BatchNorm1d(64)` qo'shing va ekvivalentlik buzilishini ko'ring.
- `lossf = nn.CrossEntropyLoss(reduction='sum')` qilib, to'plashda 1/K kerak bo'lmay qolishini tekshiring.
- K=8, 2 ishchi x 4 mikro-paket kombinatsiyasini yozing (haqiqiy DDP+accumulation).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Parallel inferens: sharding, ishchilar puli, natijalarni yig'ish

MAQSAD

Digits ustida o'qitilgan MLP og'irliklari NumPy'ga eksport qilinadi, ~360 ming so'rov 2048 talik bo'laklarga bo'linib multiprocessing Pool'ga tarqatiladi (scatter), natijalar tartib raqami bo'yicha yig'iladi (gather). 1 va 2 ishchi uchun o'tkazuvchanlik, S(N), E(N) va bo'lak kechikishining p50/p99 qiymatlari jadvalda chiqadi.

> **Kutilgan natija:** 1 ishchi ~0.5-0.6 s, 2 ishchi ~0.3 s; S(2)≈1.7-1.9, E≈85-95%; aniqlik ikkala holatda bir xil (0.993).

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Parallel inferens: ma'lumotni bo'lib yuborish (sharding), ishchilar puli,
natijalarni yig'ish
# Model — digits ustida o'qitilgan kichik MLP; inferens NumPy'da (har ishchi o'z nusxasi
bilan).
import os
os.environ["OMP_NUM_THREADS"] = "1"        # har ishchi 1 yadro — BLAS ichki parallelligi
o'chiriladi
import time, numpy as np, torch, torch.nn as nn
import multiprocessing as mp
from sklearn.datasets import load_digits

def train_model():
    torch.manual_seed(0); torch.set_num_threads(1)
    d = load_digits(); X = torch.tensor(d.data / 16.0, dtype=torch.float32); y =
torch.tensor(d.target)
    m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10))
    opt = torch.optim.Adam(m.parameters(), lr=3e-3)
    for _ in range(150):
        opt.zero_grad(); nn.functional.cross_entropy(m(X), y).backward(); opt.step()
    acc = (m(X).argmax(1) == y).float().mean().item()
    W = [(l.weight.detach().numpy().T.copy(), l.bias.detach().numpy().copy()) for l in m if
isinstance(l, nn.Linear)]
    return W, d.data / 16.0, d.target, acc

W_GLOBAL = None
def init_worker(W):
    global W_GLOBAL; W_GLOBAL = W          # fork: model nusxasi har ishchiga ko'chiriladi

def predict_shard(args):
    idx, X = args                          # idx — bo'lakning tartib raqami (natijani to'g'ri
tartibda yig'ish uchun)
    (W1, b1), (W2, b2) = W_GLOBAL
    t0 = time.perf_counter()
    h = np.maximum(X @ W1 + b1, 0); out = (h @ W2 + b2).argmax(1)
    return idx, out, time.perf_counter() - t0

if __name__ == "__main__":
    W, X, y, acc = train_model()
    # inferens yukini kattalashtiramiz: 1797 ta rasmni 200 marta takrorlab ~360k so'rov
    Xbig = np.tile(X, (200, 1)).astype(np.float32); ybig = np.tile(y, 200)
    SHARD = 2048
```

```
    shards = [(i, Xbig[s:s + SHARD]) for i, s in enumerate(range(0, len(Xbig), SHARD))]
    print(f"model aniqligi={acc:.3f}; so'rovlar={len(Xbig)}; bo'laklar={len(shards)} x
{SHARD}")
    print("=== Parallel inferens o'tkazuvchanligi ===")
    print(f"{'ishchilar':>9s} {'vaqt,s':>8s} {'so\'rov/s':>10s} {'S(N)':>6s} {'E(N)':>6s} {'p50
bo\'lak,ms':>13s} {'p99 bo\'lak,ms':>13s} {'aniqlik':>8s}")
    base = None
    for nw in (1, 2):
        ctx = mp.get_context("fork")
        with ctx.Pool(nw, initializer=init_worker, initargs=(W,)) as pool:
            pool.map(predict_shard, shards[:2])          # isitish
            t0 = time.perf_counter()
            res = pool.map(predict_shard, shards)         # scatter -> hisob -> gather
            dt = time.perf_counter() - t0
        res.sort(key=lambda r: r[0])                      # all-gather: tartib bo'yicha yig'ish
        pred = np.concatenate([r[1] for r in res]); lat = np.array([r[2] for r in res]) * 1e3
        thr = len(Xbig) / dt
        base = base or thr
        s = thr / base
        print(f"{nw:9d} {dt:8.3f} {thr:10.0f} {s:6.2f} {100*s/nw:5.0f}%
{np.percentile(lat,50):13.2f} {np.percentile(lat,99):13.2f} {(pred==ybig).mean():8.3f}")
    print("\nXulosa: inferensda qadam sayin sinxronizatsiya yo'q ('noqulay parallel'), shuning
uchun")
    print("masshtablash o'qitishga qaraganda yaxshiroq; cheklov — bo'laklarni tarqatish/yig'ish
(IPC) xarajati.")
    print("Real xizmatda esa p50/p99 kechikish va dinamik paketlash (dynamic batching) ham
o'lchanadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Parallel inferens o'tkazuvchanligi ===`

TAJRIBA QILIB KO‘RING

- `SHARD` ni 128 va 16384 qilib, IPC xarajati va p99 kechikishga ta'sirini o'lchang.
- Ishchilar sonini 4 qiling (2 yadroda) — samaradorlik nega tushadi?
- `res.sort(...)` satrini olib tashlang va `pool.imap_unordered` bilan aniqlik hisobida nima buzilishini ko'ring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Amdal va Gustafson qonunlari grafigi

MAQSAD

Parallel ulush p=0.90/0.95/0.99 uchun Amdal (kuchli masshtablash) va Gustafson (kuchsiz) tezlashish egri chiziqlari N=1..256 oralig'ida hisoblanadi va log-log grafikda chiziladi; har N uchun E(N) jadvali va $N \to \infty$ chegarasi chiqadi. Talaba o'z o'lchovlaridan (2-qadam) p ni baholab, grafikka qo'shishi mumkin.

> **Kutilgan natija:** Jadval: p=0.90 da S(256)≈9.7 (chegara 10x), p=0.99 da 72; Gustafson deyarli chiziqli. Ikki panelli grafik.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Amdal va Gustafson qonunlari: nazariy tezlashish chegaralari (grafik + jadval)
import numpy as np, matplotlib.pyplot as plt

N = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256])
fracs = [0.90, 0.95, 0.99]           # parallel qilinadigan ulush p

def amdahl(p, n):    return 1.0 / ((1 - p) + p / n)            # kuchli masshtablash (ish hajmi
o'zgarmas)
def gustafson(p, n): return (1 - p) + p * n                     # kuchsiz masshtablash (ish
hajmi N bilan o'sadi)

print("=== Amdal qonuni: S(N) = 1 / ((1-p) + p/N) ===")
print(f"{'N':>4s} " + " ".join(f"p={p:.2f}  S / E%" for p in fracs))
for n in N:
    print(f"{n:4d} " + " ".join(f"{amdahl(p, n):8.2f} /{100*amdahl(p, n)/n:4.0f}%" for p in
fracs))
print("\n=== Gustafson qonuni: S(N) = (1-p) + p*N ===")
for n in (8, 64, 256):
    print(f"N={n:3d}: " + ", ".join(f"p={p:.2f} -> S={gustafson(p, n):6.1f}" for p in fracs))

print("\nAmdal chegarasi (N -> cheksiz): " + ", ".join(f"p={p:.2f} -> {1/(1-p):.0f}x" for p in
fracs))
# Masalan: ResNet-18 DDP da aloqa ulushi ~12% (4 GPU, PCIe) -> ketma-ket ulush ~0.12 ->
maksimal ~8x
p_ddp = 1 - 0.12
print(f"Misol: aloqa ulushi 12% (ketma-ket deb olsak) -> p={p_ddp:.2f}, S(4)=
{amdahl(p_ddp,4):.2f}, E(4)={100*amdahl(p_ddp,4)/4:.0f}%, chegara {1/(1-p_ddp):.1f}x")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for p in fracs:
    ax[0].plot(N, amdahl(p, N), marker="o", label=f"p={p}")
    ax[1].plot(N, gustafson(p, N), marker="s", label=f"p={p}")
ax[0].plot(N, N, "k--", lw=1, label="ideal"); ax[1].plot(N, N, "k--", lw=1, label="ideal")
ax[0].set(xscale="log", yscale="log", title="Amdal (kuchli masshtablash)", xlabel="N
(ishchilar)", ylabel="S(N)")
ax[1].set(xscale="log", yscale="log", title="Gustafson (kuchsiz masshtablash)", xlabel="N
(ishchilar)")
for a in ax: a.grid(alpha=.3); a.legend()
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Amdal qonuni`, `=== Gustafson qonuni`

TAJRIBA QILIB KO‘RING

- 2-qadamdagi E(2) dan p ni toping: p = 2(1 - 1/S(2)) va Amdal egri chizig'iga nuqta sifatida qo'shing.
- Aloqa vaqtini N bilan o'suvchi qilib (masalan, p ulushi N oshganda kamayadi) egri chiziqning pasayishini modellashtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Aloqa/hisob nisbati kalkulyatori (GPU klasteri uchun baho)

MAQSAD

Analitik model: T_step = T_comp + (1-overlap)·T_comm, bunda T_comp FLOPs/(cho'qqi·MFU), T_comm — halqa all-reduce hajmi/tarmoq o'tkazuvchanligi. ResNet-18/50, ViT-B, GPT-2 1.5B uchun 4 xil tarmoqda r = T_comm/T_comp va E(N) hisoblanadi, gradient to'plashning ta'siri ko'rsatiladi, ResNet-50 uchun E(N) grafigi chiziladi. Talaba o'z klasteri parametrlarini (PEAK, MFU, BATCH, BYTES_PER_GRAD) kiritadi.

> **Kutilgan natija:** NVLink'da E≈99%, 10 GbE da ResNet-50 uchun E≈26% (accum=8 bilan 74%); katta model (ViT-B, GPT-2) yaxshiroq masshtablanadi.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Aloqa/hisob nisbati kalkulyatori — GPU klasteri uchun taxminiy baho (CPU'da faqat
arifmetika)
# Model: T_step = T_comp + (1 - overlap) * T_comm,  T_comm = 2(N-1)/N * S / BW,  T_comp =
FLOPs_step / (peak * MFU)
import numpy as np, matplotlib.pyplot as plt

MODELS = {   # nom: (parametrlar, FLOPs bir namuna uchun forward, GFLOP)
    "ResNet-18": (11.7e6, 1.8), "ResNet-50": (25.6e6, 4.1), "ViT-B/16": (86e6, 17.6), "GPT-2
1.5B (seq 1024)": (1.5e9, 3.0e3)}
NETS = {"NVLink 600GB/s": 600e9, "IB 400Gbit/s": 50e9, "IB 100Gbit/s": 12.5e9, "Eth 10Gbit/s":
1.25e9}
PEAK, MFU = 312e12, 0.40          # A100 BF16 cho'qqi va amaldagi foydalanish (Model FLOPs
Utilization)
BATCH, BYTES_PER_GRAD, OVERLAP, ACCUM = 128, 2, 0.5, 1   # paket/GPU, BF16 gradient, overlap
ulushi, to'plash qadamlari

def estimate(params, gflop, N, bw, batch=BATCH, bpg=BYTES_PER_GRAD, accum=ACCUM,
overlap=OVERLAP):
    t_comp = 3 * gflop * 1e9 * batch * accum / (PEAK * MFU)         # forward + backward ~ 3x
forward
    t_comm = (2 * (N - 1) / N) * params * bpg / bw if N > 1 else 0.0  # bitta all-reduce
    t_step = t_comp + (1 - overlap) * t_comm
    return t_comp, t_comm, t_comp / (t_comp + (1 - overlap) * t_comm)   # samaradorlik bahosi
E(N)

print("=== Aloqa/hisob nisbati (N=8 GPU, paket 128/GPU, BF16 gradient, overlap 50%) ===")
print(f"{'Model':24s} {'T_comp,ms':>10s} " + " ".join(f"{k[:14]:>21s}" for k in NETS))
for name, (p, g) in MODELS.items():
    cells = []
    for bw in NETS.values():
        tc, tm, e = estimate(p, g, 8, bw)
        cells.append(f"{tm*1e3:7.1f}ms r={tm/tc:5.2f} E={e*100:3.0f}%")
    print(f"{name:24s} {tc*1e3:10.2f} " + " ".join(f"{c:>21s}" for c in cells))
print("r = T_comm/T_comp (aloqa/hisob nisbati); r << 1 bo'lsa masshtablash yaxshi.")

print("\n=== Gradient to'plash ta'siri: ResNet-50, 8 GPU, 10 GbE ===")
for acc in (1, 2, 4, 8):
    tc, tm, e = estimate(*MODELS["ResNet-50"], 8, NETS["Eth 10Gbit/s"], accum=acc)
    print(f"accum={acc}: T_comp={tc*1e3:6.1f}ms  T_comm={tm*1e3:6.1f}ms  r={tm/tc:.2f}  E=
{e*100:.0f}%")
```

```
# Grafik: samaradorlik N ga bog'liq, ResNet-50, turli tarmoqlar
Ns = [1, 2, 4, 8, 16, 32, 64]
plt.figure(figsize=(7, 4))
for net, bw in NETS.items():
    plt.plot(Ns, [100 * estimate(*MODELS["ResNet-50"], n, bw)[2] for n in Ns], marker="o",
label=net)
plt.xscale("log", base=2); plt.ylim(0, 105); plt.xlabel("N GPU"); plt.ylabel("E(N), %")
plt.title("ResNet-50 DDP samaradorligi bahosi (taxminiy model)"); plt.grid(alpha=.3);
plt.legend(fontsize=8)
plt.tight_layout()
print("\nIzoh: bu analitik model; haqiqiy qiymat profiler (torch.profiler, Nsight) bilan
o'lchanadi.")
print("Kattaroq model (ViT-B) kichigidan (ResNet-18) yaxshiroq masshtablanadi — hisob tezroq
o'sadi, aloqa emas.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Aloqa/hisob nisbati`, `=== Gradient to'plash ta'siri`

TAJRIBA QILIB KO‘RING

- BYTES_PER_GRAD=4 (FP32) va 1 (FP8/siqish) uchun jadvalni qayta hisoblang.
- OVERLAP=0 va 0.9 — DDP chelak overlap'i qanchalik muhim?
- O'zingizning model (parametr soni, GFLOP) va tarmog'ingizni qo'shib, E(8) ni baholang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Epoxa va paket ta'siri** 1-qadam kodida EPOCHS=4 va BS=32, 64, 128 uchun DDP ni ishga tushiring. Har variant uchun o'tkazuvchanlik (namuna/s), oxirgi yo'qotish va aniqlikni jadvalga yozing; paket kattalashganda o'tkazuvchanlik nega oshishini 2-3 jumla bilan tushuntiring.	oson	5
2	**Aloqa hajmi jadvali** 3-qadam formulasidan foydalanib ResNet-18 (11.7 mln), ViT-B/16 (86 mln) va 7B parametrli model uchun N=2, 8, 64 GPU'da har GPU yuboradigan hajmni (FP32 va BF16) va 100 Gbit/s InfiniBand'dagi vaqtni hisoblang. Natijani 3x3 jadval ko'rinishida chiqaring va 7B model uchun DDP nega yaroqsiz ekanini izohlang.	oson	5
3	**Amdal bo'yicha p ni baholash** 2-qadamdagi o'lchovingizdan (kuchli rejim) S(2) ni olib, Amdal formulasidan ketma-ket ulush (1-p) ni toping. Shu p bilan S(4), S(8), S(64) ni hisoblang va 6-qadam grafigiga o'lchangan nuqtani qo'shing. Hisobotda p qiymati va maksimal nazariy tezlashish keltiriladi.	oson	5
4	**Kuchli vs kuchsiz masshtablash egri chizig'i** 2-qadam kodini global paket GB=64, 128, 256, 512 uchun tsiklda ishga tushiring va har GB uchun kuchli hamda kuchsiz rejimda E(2) ni o'lchang. Ikki egri chiziqni bitta matplotlib grafikda chizing (x — GB, y — E(2) %). Qaysi paketdan boshlab kuchli masshtablash «to'yinmay» qolishini aniqlang.	o'rta	10
5	**Parametr-server vs halqa** 3-qadamdagi NumPy simulyatsiyasiga parametr-server (PS) sxemasini qo'shing: barcha ishchilar gradientni 0-ishchiga yuboradi, u yig'ib qaytaradi. N=2, 4, 8, 16, 32 uchun eng band tugun yuboradigan+qabul qiladigan hajmni ikkala sxemada hisoblang, natijani jadval va grafikda ko'rsating va PS sxemasi qaysi N dan boshlab 10 GbE da 100 ms dan oshishini toping.	o'rta	10
6	**Gradient to'plash bilan DDP** 4-qadam g'oyasini 1-qadamdagi haqiqiy DDP kodiga ko'chiring: `model.no_sync()` konteksti yordamida K=4 mikro-paketdan keyin bitta all-reduce bajaring. K=1 va K=4 uchun 2 jarayonli o'qitish vaqtini va yakuniy yo'qotishni solishtiring; all-reduce chaqiruvlari soni K baravar kamayganini tushuntiring.	o'rta	10
7	**Inferens kechikish profili** 5-qadam kodida bo'lak hajmini SHARD = 64, 256, 1024, 4096, 16384 qilib, 1 va 2 ishchi uchun o'tkazuvchanlik hamda p50/p99 kechikishni o'lchang. Ikki o'qli grafik (o'tkazuvchanlik va p99) chizing va onlayn xizmat uchun (p99 < 5 ms sharti) optimal bo'lak hajmini asoslang.	o'rta	10
8	**Qo'lda DDP: all_reduce bilan** DDP o'ramisiz 2 jarayonli o'qitish yozing: 1-qadam modelida har `backward()` dan keyin har parametr gradienti uchun `dist.all_reduce(p.grad)` va `/ world_size` bajaring, boshida esa 0-rank parametrlarini `dist.broadcast` bilan tarqating. Natija	qiyin	15

	DDP varianti bilan bir xil parametrlarga (farq < 1e-6) olib kelishini ko'rsating va ikkala variant vaqtini solishtiring (DDP chelak/overlap afzalligi).		
9	**Sekin ishchi (straggler) simulyatsiyasi** 1-qadam DDP kodida rank 1 ga har qadamda `time.sleep(0.002)` qo'shib sun'iy sekinlashtiring. 2 jarayon vaqti qanday o'zgaradi va nega rank 0 ham sekinlashadi (sinxron SGD)? Keyin 3 ta turli kechikish (0, 1, 3 ms) uchun E(2) jadvalini tuzing va sekin ishchi muammosini kamaytiradigan ikkita yondashuvni (masalan, asinxron SGD, elastik o'qitish) taqqoslab yozing.	qiyin	15
10	**Klaster konfiguratsiyasi tanlovi** 7-qadam kalkulyatorini kengaytiring: berilgan model (parametr soni va GFLOP talaba tanlaydi), byudjet (GPU soni ≤ 32) va uchta tarmoq varianti uchun qadam vaqtini, E(N) ni va «1 epoxa narxi»ni (GPU-soat) hisoblaydigan funksiya yozing. Eng kam GPU-soat beradigan (N, tarmoq, accum) kombinatsiyasini toping, natijani issiqlik xaritasi (N x accum) ko'rinishida chizing va tanlovni asoslang.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Telekom: abonent segmentlari tasnifi — 2 jarayonli DDP va paket ta'siri

oson

Soha: Telekom (mobil operator)

Mobil operator abonentlarning kunlik chaqiruv va trafik yozuvlaridan (CDR) 6 ta tarif segmentini bashorat qiluvchi MLP'ni 2 GPU'li serverda o'qitmoqchi. Sandbox'da GPU yo'q, shuning uchun 2 CPU yadro va gloo backend bilan DDP mantig'i aynan shu tarzda tekshiriladi. Talab: har rank ma'lumotning kesishmaydigan yarmini ko'rishi, parametrlar sinxron bo'lishi (farq 0), o'tkazuvchanlik uchta paket hajmi uchun o'lchanishi va 10 Gb/s tarmoqda aloqa vaqti baholanishi.

Ma'lumot	sintetik CDR: make_classification(n=6000, 40 belgi, n_informative=20, 6 sinf, random_state=7)
Model	MLP 40-192-192-6 (taxminan 46 ming parametr)
Paket (har rank)	32, 64, 128
Epoxa	3
LR qoidasi	0.03 * world_size, momentum 0.9
Tarmoq (kalkulyator)	10 Gb/s
Ishchilar	2 jarayon, gloo, 1 yadro/jarayon

QADAMLAR

1. Laboratoriya s1 kodini make_classification ma'lumotiga moslang (X ni float32, standartlang). BS=32, 64, 128 va EPOCHS=3 uchun DDP'ni ishga tushiring; har holat uchun jami ko'rilgan namunalar (36000 bo'lishi kerak), namuna/s, oxirgi yo'qotish, aniqlik va ranklar orasidagi parametr farqini jadvalga yozing.
2. s2 kabi HID=192, GB=128, STEPS=60 bilan 4 konfiguratsiyani (1 jarayon; DDP kuchli; DDP kuchsiz; 1 jarayon 2 yadro) o'lchang, S(2) va E(2) ni hisoblang.
3. s3 ring all-reduce funksiyasini N=2, 4, 8 uchun tekshiring (natija to'g'ri=True), so'ng o'z modelingiz gradienti (46 ming parametr, FP32) va ResNet-18 (11.7 mln) uchun 1, 10, 100 Gb/s tarmoqlarda bitta all-reduce vaqtini N=2, 8, 32 da hisoblang.
4. s4 kabi 96 namunali global paket uchun 1x96, 3x32 (to'plash, 1/K bilan va 1/K siz) va 2 ishchi x 48 gradientlarini solishtiring; max farqlarni chiqaring.
5. s5 kabi o'qitilgan modelni NumPy'ga eksport qilib, 300 ming sintetik CDR so'rovini SHARD=1024 bilan 1 va 2 ishchida bajaring; so'rov/s, S(2), E(2), p50/p99 bo'lak kechikishini jadvalga kiriting.
6. 2-qadamdagi kuchli rejim S(2) dan Amdal bo'yicha p ni baholang, S(8) va S(32) hamda chegarani hisoblang; s6 grafigiga o'lchangan nuqtani qo'shing.
7. Xulosa: paket kattalashganda o'tkazuvchanlik nega oshadi, 10 Gb/s tarmoq shu model uchun to'siq bo'ladimi, 2 GPU serverga qaysi paketni tavsiya qilasiz.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun

tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

2-variant. Agro: tuproq namligi sensorlari — kuchli va kuchsiz masshtablash

oson

Soha: Agro

Agroklaster 8 ta tuproq sensorining 24 soatlik o'lchovlari (192 belgi) bo'yicha dalaning sug'orish holatini 4 sinfga (shoshilinch, bugun, 2 kundan keyin, kerak emas) ajratuvchi MLP'ni fermer xo'jaligi serverida (2 ta GPU, tugunlar orasida 1 Gb/s tarmoq) o'qitmoqchi. Sizdan: CPU sandbox'da kuchli va kuchsiz masshtablashni ikki model o'lchami uchun o'lchash va 1 Gb/s tarmoqda aloqa vaqtini baholab, qaysi rejim maqsadga muvofiqligini aytish talab qilinadi.

Ma'lumot	sintetik sensor matritsasi: 8000 x 192 (sinus kunlik sikl + shovqin), 4 sinf
Model	MLP 192-HID-HID-4, HID = 256 va 768
Global paket	128 va 256
STEPS	50
Tarmoq	1 Gb/s (tugunlar orasida), NVLink muqobili yo'q
Gradient formati	FP32 va BF16

QADAMLAR

1. Sensor generatorini yozing: har sensor uchun kunlik sinus + tasodifiy shovqin, sinf namlik o'rtachasi bo'yicha; s1 DDP kodini shu ma'lumotga moslang, EPOCHS=2, BS=64; parametr farqi 0 ekanini ko'rsating.
2. s2 kodini HID=256 va HID=768, GB=128 va 256 uchun (4 kombinatsiya) ishga tushiring; har biri uchun T(1), kuchli va kuchsiz DDP vaqtini, S(2), E(2) ni jadvalga yozing.
3. E(2) ni HID bo'yicha ikki rejimda ustunli diagrammada chizing; kichik modelda kuchli rejim samaradorligi nega past ekanini izohlang.
4. s3 formulasidan HID=768 modeli gradient hajmini (parametrlar sonini kod bilan hisoblang) FP32 va BF16 da toping va 1 Gb/s tarmoqda N=2, 4, 8 uchun all-reduce vaqtini hisoblang; ring simulyatsiyasini N=4 da tekshiring.
5. s4 kabi gradient to'plash K=2 va K=4 ekvivalentligini 128 namunali paketda tekshiring (farq < 1e-6).
6. s6 Gustafson formulasi bilan kuchsiz rejim uchun S(N) ni N=2..64 da chizing, o'lchangan E(2) dan p ni baholang.
7. Xulosa: fermer xo'jaligi klasteri uchun kuchsiz masshtablash + BF16 + gradient to'plash kombinatsiyasini raqamlar bilan asoslang.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

3-variant. Moliya: firibgarlik detektori gradientlari uchun halqa all-reduce va tarmoq tanlovi oson

Soha: Moliya (to'lov tizimi)

To'lov tizimi uchta nomzod modelni ko'rib chiqmoqda: kichik MLP (2.1 mln parametr), jadval-transformer (45 mln) va matnli izohlar uchun 7 mlrd parametrli til modeli. Infratuzilma bo'limi 1, 10 va 100 Gb/s tarmoq variantlarini taklif qiladi. Sizdan har model va tarmoq uchun bitta all-reduce vaqtini hisoblash, NumPy ring all-reduce simulyatsiyasini tekshirish va qaysi model DDP bilan umuman masshtablanmasligini ko'rsatish talab qilinadi.

Modellar	MLP 2.1 mln; TabTransformer 45 mln; LLM 7 mlrd parametr
Tarmoqlar	1, 10, 100 Gb/s
N (GPU)	2, 4, 8, 16, 64
Gradient formati	FP32 (4 bayt) va BF16 (2 bayt)
Simulyatsiya vektori	L = 2400 element, N = 2, 3, 6
Hisob vaqti (taqqoslash)	T_comp = 120 ms/qadam

QADAMLAR

1. s3 ring_allreduce funksiyasini L=2400 va N=2, 3, 6 (N ga bo'linmaydigan uzunlik!) uchun ishga tushiring; natija to'g'ri=True va yuborilgan baytlar 2(N-1)/N*S formulasiga mos ekanini ko'rsating.
2. Funksiyani SUM/N (o'rtacha) qaytaradigan qilib o'zgartiring va DDP shu rejimda ishlashini izohlang.
3. Uchta model x uchta tarmoq x N=2,4,8,16,64 uchun har GPU yuboradigan hajm (MB) va aloqa vaqti (ms) jadvalini FP32 da chiqaring; BF16 uchun takrorlang.
4. T_comp=120 ms deb olib, r = T_comm/T_comp va E(N) = T_comp/(T_comp+T_comm) ni hisoblang; E(8) < 50% bo'lgan kombinatsiyalarni belgilang.
5. s1 DDP kodini make_classification(n=5000, 30 belgi, 2 sinf, weights=[0.97, 0.03]) firibgarlik ma'lumotiga moslang va 2 jarayonda parametr sinxronligini tekshiring (DDP mantig'i kichik modelda).
6. Xulosa: 7 mlrd model uchun DDP nega yaroqsiz (xotira va 2S hajm), FSDP/ZeRO qanday yordam beradi, 45 mln model uchun minimal tarmoq qaysi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

4-variant. Tibbiyot: EKG segmentlari tasnifi — gradient to'plash ekvivalentligi

oson

Soha: Tibbiyot (kardiologiya)

Kardiologiya markazi 256 nuqtali EKG segmentlarini 3 sinfga (normal, aritmiya, artefakt) ajratuvchi modelni o'qitadi. Klinika serverida xotira kam, shuning uchun 192 namunali global paketni bir yo'la sig'dirib bo'lmaydi — gradient to'plash yoki 2 GPU'da ma'lumotlar parallelligi kerak. Sizdan to'plash, DDP o'rtachalash va bitta katta paket bir xil gradient berishini isbotlash, xato variantlarni (1/K siz, BatchNorm bilan) ko'rsatish talab qilinadi.

Ma'lumot	sintetik EKG: 256 nuqta, sinus + QRS impuls + shovqin, 3 sinf, n=3000
Model	MLP 256-96-3 (BatchNorm bilan va BatchNorm'siz)
Global paket	192
K (mikro-paket)	3 va 6
Ishchilar (qo'lda DDP)	2 x 96 va 4 x 48
Loss reduction	mean va sum

QADAMLAR

1. EKG generatorini yozing (har sinf uchun boshqa impuls shakli); s4 kodini 192 namunali paketga moslang: (A) 1x192, (B) K=3 to'plash 1/K bilan, (C) 2 ishchi x 96 o'rtacha; max farqlarni chiqaring (< 1e-6).
2. K=6 va 4 ishchi x 48 variantlarini qo'shing; (B') 1/K siz to'plashda ||gE||/||gA|| nisbati K ga teng ekanini ko'rsating.
3. Modelga nn.BatchNorm1d(96) qo'shing va ekvivalentlik buzilishini raqam bilan ko'rsating; nega buzilishini izohlang (SyncBatchNorm haqida 2 jumla).
4. reduction='sum' bilan to'plashda 1/K kerak emasligini tekshiring va LR qanday o'zgarishi kerakligini yozing.
5. s1 DDP kodini shu EKG ma'lumotiga moslang (BS=48, EPOCHS=3) va 2 jarayonda parametr farqi 0 ekanini ko'rsating; namuna/s ni o'lchang.
6. s3 formulasi bilan model gradienti (parametr sonini hisoblang) uchun 1 Gb/s klinika tarmog'ida all-reduce vaqtini N=2 da toping va to'plash K=6 aloqa chastotasini qanday kamaytirishini jadvalda ko'rsating.
7. Xulosa: xotira cheklangan klinika serverida qaysi usulni (to'plash yoki DDP) tavsiya qilasiz va nima uchun.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

5-variant. Chakana savdo: mahsulot toifasini aniqlash inferensi — ishchilar puli

oson

Soha: Chakana savdo

Onlayn do'kon har kecha 500 ming yangi mahsulot kartochkasini 12 toifaga avtomatik ajratadi. Hozir bu bitta jarayonda 40 daqiqa olmoqda; rahbariyat 2 va 4 yadroli serverda parallel inferens qanchalik tezlashtirishini bilmoqchi. Sizdan multiprocessing Pool bilan sharding, natijalarni tartib bilan yig'ish va bo'lak hajmining o'tkazuvchanlik hamda kechikishga ta'sirini o'lchash talab qilinadi.

Ma'lumot	make_classification(n=5000, 50 belgi, 12 sinf, n_informative=30, random_state=3), 100 marta takrorlab 500 ming so'rov
Model	MLP 50-128-12, Adam, 200 qadam
SHARD	512, 2048, 8192
Ishchilar	1, 2 (va 4 — 2 yadroda sinov)
SLO	p99 bo'lak kechikishi < 8 ms
Yig'ish	tartiblangan map va imap_unordered

QADAMLAR

1. s5 kodini make_classification ma'lumotiga moslang: modelni o'qiting (aniqlikni yozing), og'irliklarni NumPy'ga eksport qiling, 500 ming so'rovni tayyorlang.
2. SHARD=2048 bilan 1 va 2 ishchi uchun so'rov/s, S(2), E(2), p50/p99 bo'lak kechikishini o'lchang; aniqlik ikkala holatda bir xil ekanini tekshiring.
3. SHARD=512 va 8192 uchun takrorlang; 3x2 jadval tuzing va ikki o'qli grafik (so'rov/s va p99) chizing; SLO p99 < 8 ms ni qondiruvchi eng katta bo'lakni tanlang.
4. 4 ishchini 2 yadroda ishga tushiring — E(4) nega 50% atrofida yoki past bo'lishini izohlang.
5. pool.imap_unordered bilan res.sort'siz yig'ing va aniqlik qanday buzilishini ko'rsating; so'ng idx bo'yicha tiklang.
6. s6 Amdal formulasi bilan o'lchangan S(2) dan p ni baholang va 8 yadroli server uchun S(8) ni bashorat qiling; 40 daqiqa qancha bo'lishini hisoblang.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

6-variant. Ta'lim: o'quvchi natijalarini bashorat qiluvchi klaster — Amdal va Gustafson oson

Soha: Ta'lim

Respublika ta'lim portali 2 mln o'quvchi yozuvi bo'yicha o'zlashtirish xavfini (3 daraja) bashorat qiluvchi modelni o'qitadi va IT bo'limi 4, 16 yoki 64 GPU sotib olish masalasini hal qilmoqda. Sizdan CPU'da 2 jarayonli o'lchov o'tkazib, ketma-ket ulushni baholash va Amdal hamda Gustafson qonunlari asosida har uchala klaster hajmi uchun kutilayotgan tezlashishni hisoblab berish talab qilinadi.

Ma'lumot	sintetik o'quvchi yozuvlari: 12000 x 24 belgi (davomat, baholar, ijtimoiy ko'rsatkichlar), 3 sinf
Model	MLP 24-256-256-3
p (nazariy)	0.80, 0.92, 0.98
N	1, 2, 4, 8, 16, 32, 64, 128, 512
Global paket (kuchli)	192
STEPS	60

QADAMLAR

1. s1 DDP kodini o'quvchi ma'lumotiga moslang (EPOCHS=2, BS=96) va 2 jarayonda to'g'ri ishlashini (parametr farqi 0, jami 24000 namuna) ko'rsating.
2. s2 kodi bilan HID=256, GB=192, STEPS=60 uchun kuchli va kuchsiz rejimda S(2), E(2) ni o'lchang.
3. Kuchli rejim S(2) dan Amdal bo'yicha ketma-ket ulushni toping: 1-p = (2/S(2) - 1); p qiymatini yozing.
4. s6 kodini p=0.80, 0.92, 0.98 va o'lchangan p uchun N=1..512 oralig'ida ishga tushiring; S(N), E(N) jadvali va log-log grafik (o'lchangan nuqta bilan).
5. Gustafson formulasi bilan kuchsiz rejim uchun S(4), S(16), S(64) ni hisoblang; ikki qonun farqini bitta grafikda ko'rsating.
6. Xulosa: 4, 16, 64 GPU uchun kutilayotgan E(N) jadvali; Amdal bo'yicha 64 GPU sotib olish o'zini oqlaydimi, kuchsiz rejimda-chi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

7-variant. Energetika: elektr yuklama prognozi — aloqa/hisob kalkulyatori (V100 klasteri) oson

Soha: Energetika

Energiya kompaniyasi 3 ta vaqt qatori modelini (kichik LSTM, TCN, Transformer) 8 ta V100 GPU'li klasterda o'qitmoqchi; tugunlar orasidagi tarmoq 1, 10 yoki 100 Gb/s bo'lishi mumkin. Sizdan s7 analitik modeli bilan har model va tarmoq uchun T_comp, T_comm, r va E(8) ni hisoblash, gradient to'plash ta'sirini ko'rsatish va byudjet doirasida tarmoq tanlash talab qilinadi.

Modellar	LSTM-kichik 2.3 mln / 0.5 GFLOP; TCN 9 mln / 1.2 GFLOP; Transformer 110 mln / 22 GFLOP (bir namuna forward)
GPU	V100 FP16 cho'qqi 125 TFLOPS, MFU 0.35
Tarmoqlar	1, 10, 100 Gb/s
Paket/GPU	64
Gradient	BF16 (2 bayt)
Overlap	0.5
Accum	1, 2, 4, 8

QADAMLAR

1. s7 kodidagi MODELS, NETS, PEAK, MFU, BATCH ni variant qiymatlariga almashtiring va N=8 uchun 3x3 jadvalni (T_comm, r, E) chiqaring.
2. Transformer uchun accum=1, 2, 4, 8 da 10 Gb/s tarmoqda E(8) ni hisoblang va to'plash qaysi qiymatdan boshlab E > 80% berishini toping.
3. Har tarmoq uchun N=1..64 da E(N) egri chiziqlarini TCN modeli uchun chizing (log2 o'q).
4. BYTES_PER_GRAD=4 (FP32) va OVERLAP=0 holatlarini qo'shib, eng yomon va eng yaxshi holat jadvalini tuzing.
5. s3 ring simulyatsiyasini N=8 da tekshiring va LSTM-kichik gradient hajmi uchun 1 Gb/s da haqiqiy all-reduce vaqtini formula bilan solishtiring.
6. Xulosa: 10 Gb/s tarmoq qaysi modellar uchun yetarli, 100 Gb/s qachon zarur, kichik LSTM uchun 8 GPU umuman kerakmi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

8-variant. Transport: yo'l tirbandligi bashorati — sekin ishchi (straggler) ta'siri

o'rta

Soha: Transport

Shahar transport markazi 300 ta yo'l sensoridan kelayotgan oqim bo'yicha 15 daqiqalik tirbandlik darajasini (5 sinf) bashorat qiluvchi modelni 2 GPU'da o'qitadi, lekin GPU'lardan biri boshqa xizmat bilan birga ishlaydi va vaqti-vaqti bilan sekinlashadi. Sizdan sinxron SGD'da bitta sekin ishchi butun o'qitishni qanday tutib qolishini o'lchash va kechikish darajasiga qarab E(2) jadvalini tuzish, so'ng yumshatish usullarini taqqoslash talab qilinadi.

Ma'lumot	sintetik sensor oqimi 10000 x 300, 5 sinf (soat va hafta kuni bo'yicha naqsh)
Model	MLP 300-384-384-5
Sun'iy kechikish (rank 1)	0, 1, 2, 4 ms har qadamda
BS (har rank)	64
EPOCHS	2
Qadam vaqti (baza)	o'lchanadi
Tarmoq (kalkulyator)	10 Gb/s

QADAMLAR

1. s1 DDP kodini transport ma'lumotiga moslang va kechikishsiz baza vaqtini hamda bitta qadam o'rtacha vaqtini (ms) o'lchang.
2. rank 1 ga har qadamda time.sleep(d) qo'shing (d = 1, 2, 4 ms); har d uchun ikkala rank vaqtini, namuna/s va E(2) = T_baza/T_d ni jadvalga yozing. Rank 0 nega ham sekinlashadi — all-reduce to'siq (barrier) tushunchasi bilan izohlang.
3. Kechikishning qadam vaqtiga nisbati (d / T_qadam) bo'yicha E(2) grafigini chizing va chiziqli model E = 1/(1 + d/T_qadam) bilan solishtiring.
4. s2 kodi bilan kechikishsiz kuchli/kuchsiz S(2) ni o'lchang va straggler ta'sirini shu jadvalga qo'shimcha ustun qiling.
5. s3 formulasi bilan model gradienti hajmini va 10 Gb/s da aloqa vaqtini hisoblang; aloqa vaqti bilan 4 ms kechikishni taqqoslang.
6. s6 Amdal modeliga straggler'ni ketma-ket ulush sifatida qo'shing va N=8 da kutilayotgan E(8) ni baholang.
7. Xulosa: asinxron SGD, zaxira ishchilar (backup workers) va elastik o'qitishning afzallik/kamchiliklarini jadvalda taqqoslang va transport markazi uchun tanlov bering.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot

sifati (grafik, tuzilma, til) — 20

9-variant. Logistika: ombor buyurtmalarini marshrutlash inferensi — bo'lak hajmi va p99 o'rta

Soha: Logistika

Katta ombor har soatda 1.2 mln buyurtma qatorini 16 ta yig'ish zonasiga taqsimlovchi modelni ishlatadi; onlayn rejimda har bo'lak uchun p99 kechikish 4 ms dan oshmasligi kerak, kechki qayta ishlashda esa maksimal o'tkazuvchanlik muhim. Sizdan bo'lak hajmi bo'yicha o'tkazuvchanlik va p50/p99 profilini 1 va 2 ishchida o'lchash, ikki rejim uchun turli optimal bo'lakni asoslash talab qilinadi.

Ma'lumot	make_classification(n=6000, 36 belgi, 16 sinf, n_informative=24, random_state=11), 200 marta takror (1.2 mln so'rov)
Model	MLP 36-160-16
SHARD	64, 256, 1024, 4096, 16384
Ishchilar	1, 2
SLO onlayn	p99 bo'lak < 4 ms
Maxsus	imap chunksize ta'siri

QADAMLAR

1. s5 kodini moslang: modelni o'qiting, 1.2 mln so'rov tayyorlang; SHARD ro'yxati bo'yicha 1 va 2 ishchida so'rov/s, S(2), E(2), p50, p99 ni 5x2 jadvalga yozing.
2. Ikki o'qli grafik: x — SHARD (log), chap o'q — so'rov/s, o'ng o'q — p99 ms; 1 va 2 ishchi egri chiziqlari.
3. Onlayn rejim (p99 < 4 ms) va kechki rejim (maks. so'rov/s) uchun optimal SHARD ni tanlang va asoslang.
4. pool.map o'rniga pool.imap(..., chunksize=c) bilan c=1, 4, 16 uchun kichik SHARD=64 da IPC xarajati qanday kamayishini o'lchang.
5. 4 ishchi 2 yadroda: E(4) ni o'lchang va kontekst almashinuvi ta'sirini izohlang.
6. s6 Amdal bilan eng yaxshi S(2) dan p ni baholang va 8 yadroli server uchun soatlik 1.2 mln buyurtma necha daqiqada bajarilishini bashorat qiling.
7. Xulosa: ikki rejim uchun konfiguratsiya jadvali (SHARD, ishchilar, kutilgan p99, so'rov/s) va dinamik paketlash haqida tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

10-variant. Sanoat: dastgoh nosozligini bashorat — masshtablash egri chizig'i (paket va model o'lchami) o'rta

Soha: Sanoat

Mashinasozlik zavodi 120 ta vibratsiya belgisidan dastgoh nosozligini 7 sinfga ajratuvchi modelni o'qitadi va 2 GPU'li stansiyada kuchli masshtablash qachon to'yinishini bilmoqchi. Sizdan global paket va model kengligi bo'yicha E(2) egri chiziqlarini ikki rejimda o'lchash, to'yinish nuqtasini aniqlash va bucket_cap_mb ta'sirini tekshirish talab qilinadi.

Ma'lumot	sintetik vibratsiya spektrlari 8000 x 120, 7 sinf
Model	MLP 120-HID-HID-7, HID = 256 va 1024
Global paket	32, 64, 128, 256, 512
STEPS	40
DDP sozlamasi	bucket_cap_mb = 1 va 25 (standart)
Tarmoq (kalkulyator)	100 Gb/s

QADAMLAR

1. s2 kodini tsiklga o'rab, HID=256 uchun GB ro'yxati bo'yicha T(1), kuchli va kuchsiz DDP vaqtini o'lchang; E(2) ni ikki rejimda hisoblang (jami vaqt 90 s dan oshsa STEPS ni 25 ga tushiring).
2. HID=1024 uchun takrorlang (kerak bo'lsa GB ro'yxatini 64..512 qiling).
3. Bitta grafikda 4 egri chiziq: (HID, rejim) bo'yicha E(2) vs GB; kuchli rejim qaysi GB dan boshlab to'yinishini (E > 85%) har HID uchun aniqlang.
4. DDP(model, bucket_cap_mb=1) bilan HID=1024, GB=256 ni qayta o'lchang va standart bilan solishtiring; chelak soni ko'payganda nima o'zgaradi.
5. s1 kodini vibratsiya ma'lumotiga moslang (EPOCHS=2) va parametr sinxronligi hamda aniqlikni ko'rsating.
6. s3 formulasi bilan HID=1024 model gradienti uchun 100 Gb/s da all-reduce vaqtini va s7 modeli bilan r nisbatini baholang; o'lchangan E(2) bilan mos keladimi.
7. Xulosa: zavod stansiyasi uchun GB va rejim tavsiyasi, to'yinish nuqtasi va chiziqli LR qoidasi haqida eslatma.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

11-variant. Video-kuzatuv: kadr embedding tasnifi — parametr-server va halqa taqqoslovi o'rta

Soha: Xavfsizlik

Xavfsizlik kompaniyasi 2048 o'lchamli kadr embedding'laridan 9 ta hodisa turini aniqlovchi modelni ko'p GPU'li klasterda o'qitadi; arxitektura guruhi parametr-server (PS) va halqa all-reduce sxemalari orasida tanlov qilmoqda, tugunlar orasida 10 Gb/s tarmoq bor. Sizdan NumPy'da ikkala sxemani yozish, eng band tugun trafigini N bo'yicha solishtirish va PS qaysi N dan boshlab 100 ms chegarasidan oshishini topish talab qilinadi.

Ma'lumot	sintetik embedding 6000 x 2048 (9 ta markaz + shovqin), 9 sinf
Model (gradient hajmi)	embedding tasniflagich 2048-512-9 (taxminan 1.05 mln) va ResNet-50 23 mln
N	2, 4, 8, 16, 32, 64
Tarmoq	10 Gb/s
Chegara	100 ms / all-reduce
Simulyatsiya	L = 4096, N = 2, 4, 8

QADAMLAR

1. s3 ring_allreduce ni L=4096, N=2,4,8 da tekshiring; so'ng parametr-server sxemasini yozing (hamma 0-ishchiga yuboradi, u yig'ib hammaga qaytaradi) va natija bir xil ekanini ko'rsating.
2. Har ikki sxema uchun eng band tugun yuboradigan + qabul qiladigan baytlarni N ro'yxati bo'yicha jadvalga yozing (S = model hajmi FP32, ikkala model uchun).
3. 10 Gb/s da vaqtni hisoblang; PS sxemasi ResNet-50 uchun qaysi N dan 100 ms dan oshadi; halqa uchun chegara 2S/BW ekanini grafikda ko'rsating (log-log).
4. s1 DDP kodini embedding ma'lumotiga moslang (BS=64, EPOCHS=2) va parametr sinxronligini tekshiring; DDP ichida qaysi sxema ishlatilishini izohlang.
5. s2 bilan HID=512 uchun S(2), E(2) ni o'lchang.
6. s7 kalkulyatoriga PS sxemasini qo'shing (T_comm_PS = 2(N-1)*S/BW server uchun) va ResNet-50, N=8 da E(8) ni ikki sxemada solishtiring.
7. Xulosa: qaysi N gacha PS maqbul, halqa all-reduce'ning kamchiligi (kechikish, straggler) va kompaniya uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

12-variant. Media: yangiliklar matnini tasniflash — no_sync bilan gradient to'plash DDP'da o'rta

Soha: Media

Yangiliklar agentligi 3000 o'lchamli so'z-sumka vektorlaridan 8 ta rubrikani aniqlovchi modelni 2 GPU'da o'qitadi; tarmoq sekin bo'lgani uchun har qadamda all-reduce qilish qimmat. Sizdan haqiqiy DDP kodida model.no_sync() konteksti yordamida K mikro-paketdan keyin bitta all-reduce bajarish, K bo'yicha vaqt va yakuniy yo'qotishni o'lchash hamda all-reduce soni K baravar kamayganini ko'rsatish talab qilinadi.

Ma'lumot	sintetik so'z-sumka: 6000 x 3000 (siyrak, 8 rubrika markazi), float32
Model	MLP 3000-256-8 (taxminan 770 ming parametr)
K (to'plash)	1, 2, 4, 8
Mikro-paket (har rank)	32
EPOCHS	2
Tarmoq (kalkulyator)	1 Gb/s
Tekshiruv	all-reduce soni = qadamlar / K

QADAMLAR

1. s1 DDP kodini so'z-sumka ma'lumotiga moslang (BS=32, EPOCHS=2) va K=1 baza vaqtini, namuna/s va yakuniy yo'qotishni o'lchang.
2. O'qitish siklini qayta yozing: K-1 mikro-paket model.no_sync() ichida backward, K-chisi oddiy backward + opt.step(); yo'qotishni 1/K ga ko'paytiring. K=2, 4, 8 uchun vaqt, namuna/s, yakuniy yo'qotish jadvali.
3. All-reduce chaqiruvlari sonini hisoblang (DDP hook yoki hisoblagich bilan) va K baravar kamayganini ko'rsating; chiziqli LR qoidasi global paket K*32*2 uchun qanday bo'lishi kerakligini yozing.
4. s4 kabi K=4 to'plash + 2 ishchi kombinatsiyasining bitta 256 namunali paket bilan gradient ekvivalentligini tekshiring (farq < 1e-6).
5. s3 formulasi bilan 770 ming parametr gradienti uchun 1 Gb/s da bitta all-reduce vaqtini hisoblang va K bo'yicha qadamga to'g'ri keladigan aloqa vaqtini jadvalga qo'shing.
6. s6 Amdal modelida aloqa ulushini K ga bo'lib, N=8 uchun kutilgan E(8) ni K=1 va K=8 da baholang.
7. Xulosa: K ning ortishi qaysi nuqtadan boshlab yaqinlashuvga zarar bera boshlaydi (yo'qotish jadvali), agentlik uchun tavsiya etilgan K.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot

sifati (grafik, tuzilma, til) — 20

13-variant. Aqlli shahar: kamera oqimlari uchun klaster kalkulyatori (1/10/100 Gb/s) o'rta

Soha: Shahar

Shahar boshqaruv markazi 400 ta kameradan kelayotgan kadrlarni tahlil qiluvchi uchta modelni (kichik YOLO, BERT-base matn uchun, ViT-L) 16 GPU'li A100 klasterida o'qitishni rejalashtiradi; tarmoq uchun 1, 10 va 100 Gb/s variantlari va overlap darajasi noma'lum. Sizdan s7 kalkulyatorini kengaytirib, N=4, 8, 16 da E(N) ni hisoblash, overlap va gradient formatining ta'sirini ko'rsatish va har model uchun minimal yetarli tarmoqni tanlash talab qilinadi.

Modellar	YOLO-kichik 7 mln / 8 GFLOP; BERT-base 110 mln / 11 GFLOP (seq 128); ViT-L 304 mln / 61 GFLOP
GPU	A100 BF16 312 TFLOPS, MFU 0.4
Tarmoqlar	1, 10, 100 Gb/s
N	4, 8, 16
Overlap	0.3 va 0.7
Gradient	FP32, BF16
Paket/GPU	32

QADAMLAR

1. s7 kodidagi MODELS, NETS, BATCH, OVERLAP ni variant qiymatlariga almashtiring; N=8, overlap=0.5 uchun 3x3 jadval (T_comm, r, E).
2. Har model uchun N=4, 8, 16 x 3 tarmoq E(N) jadvalini tuzing va E > 80% shartini qondiruvchi minimal tarmoqni belgilang.
3. Overlap 0.3 va 0.7 uchun ViT-L, 10 Gb/s da E(16) ni solishtiring; DDP chelaklari overlap'ni qanday ta'minlashini izohlang.
4. BYTES_PER_GRAD=4 va 2 uchun BERT-base natijasini solishtiring; gradient siqish 1 Gb/s da yetarli bo'ladimi.
5. Accum=1,2,4,8 uchun YOLO-kichik, 1 Gb/s da E(16) ni hisoblang va grafik chizing.
6. s3 ring simulyatsiyasini N=16 da tekshiring va YOLO-kichik hajmi uchun har GPU yuboradigan baytlar 2S ga yaqinlashishini ko'rsating.
7. Xulosa: har model uchun (tarmoq, accum, gradient formati) tavsiyasi va umumiy klaster uchun bitta tarmoq tanlansa qaysi biri.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

14-variant. Ekologiya: sun'iy yo'ldosh tasvirlarini tasniflash — gradient siqish (FP32/BF16/FP8) o'rta

Soha: Ekologiya

Ekologik monitoring markazi 512 o'lchamli sun'iy yo'ldosh tasvir belgilaridan 6 ta yer qoplami turini aniqlovchi modelni 4 ta viloyat serverida (tugunlar orasida 1 Gb/s) o'qitmoqchi; aloqa to'siq bo'lgani uchun gradientlarni BF16 yoki FP8 ga siqish ko'rib chiqilmoqda. Sizdan NumPy ring all-reduce'da past aniqlikli formatlarning hajm va xatolik ta'sirini o'lchash, aloqa vaqtini hisoblash va DDP'da xatolik to'planishini baholash talab qilinadi.

Ma'lumot	sintetik tasvir belgilari 6000 x 512, 6 sinf
Model	MLP 512-384-6 (taxminan 199 ming parametr)
Gradient formati	FP32, BF16 (torch.bfloat16 orqali), FP8-taqlid (int8 kvantlash, masshtab bilan)
Tarmoq	1 Gb/s
N	2, 4, 8, 16
Xatolik o'lchovi	nisbiy L2 xato, kosinus o'xshashlik

QADAMLAR

1. s3 ring_allreduce ga 'dtype' parametri qo'shing: yuborishdan oldin bo'lakni BF16 (torch orqali) yoki int8 (max-abs masshtab) ga aylantirib, qabulda qaytaring; N=4, L=8192 da uch format uchun yuborilgan baytlar va natijaning aniq yig'indiga nisbiy xatosini jadvalga yozing.
2. Formula bilan 199 ming parametr modeli va ResNet-50 (25.6 mln) uchun 1 Gb/s da N=2,4,8,16 all-reduce vaqtini uch formatda hisoblang.
3. s4 gradient ekvivalentligi kodida 2 ishchi gradientini BF16 va int8 ga siqib o'rtachalang; max farq va kosinus o'xshashlikni FP32 bilan solishtiring.
4. s1 DDP kodini tasvir belgilariga moslang va DDP'ga BF16 siqish hook'i (register_comm_hook, default_hooks.bf16_compress_hook) qo'shing; vaqt va yakuniy aniqlikni hook'siz holat bilan solishtiring.
5. s7 kalkulyatorida BYTES_PER_GRAD = 4, 2, 1 uchun ResNet-50, 1 Gb/s, N=4 da E(4) ni hisoblang.
6. Xulosa: qaysi format aniqlik yo'qotmasdan aloqa vaqtini yetarlicha kamaytiradi, xatolik-fikr (error feedback) nima uchun kerak, viloyat klasteri uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

15-variant. Sport: harakat sensorlari bo'yicha mashq turini aniqlash — DDP'ni qo'lda yozish o'rta

Soha: Sport

Sport texnologiyalari startapi bilak sensorlaridan (6 o'q x 50 nuqta = 300 belgi) 10 ta mashq turini aniqlovchi modelni o'qitadi va jamoa DDP o'rami ichida nima bo'layotganini tushunishi kerak. Sizdan DDP'siz 2 jarayonli o'qitishni qo'lda yozish (broadcast + har gradient uchun all_reduce), natijasi DDP bilan bir xil ekanini isbotlash va vaqt farqini (chelak/overlap afzalligi) o'lchash talab qilinadi.

Ma'lumot	sintetik IMU signallari 5000 x 300 (10 mashq uchun turli chastotali sinuslar), 10 sinf
Model	MLP 300-256-128-10
BS (har rank)	64
EPOCHS	2
Taqqoslash	DDP o'rami vs qo'lda broadcast + all_reduce(p.grad)/world
Tekshiruv	parametr farqi < 1e-6, bir xil urug' va sampler

QADAMLAR

1. s1 DDP kodini IMU ma'lumotiga moslang va DDP bilan yakuniy parametrlarni (flat vektor) faylga saqlang; vaqtni o'lchang.
2. Qo'lda variant yozing: DDP o'ramisiz, boshida har parametr uchun dist.broadcast(p.data, src=0); har backward'dan keyin har p.grad uchun dist.all_reduce(SUM) va /world; sampler va urug' bir xil bo'lsin.
3. Ikkala variant yakuniy parametrlarini solishtiring (max farq < 1e-6) va vaqtni jadvalga yozing; DDP tezroq bo'lsa — chelaklar va overlap tufayli ekanini izohlang.
4. Qo'lda variantda gradientlarni bitta flat vektorga yig'ib (torch.cat) bitta all_reduce qiling; vaqt qanday o'zgaradi — all-reduce soni va kechikish (latency) rolini tushuntiring.
5. s3 formulasi bilan model gradienti hajmini va 10 Gb/s da all-reduce vaqtini hisoblang; parametrlar soni bo'yicha chaqiruvlar soni (qatlam soni) bilan solishtiring.
6. s2 kodi bilan S(2), E(2) ni DDP va qo'lda (flat) variant uchun o'lchang va jadvalga kiriting.
7. Xulosa: DDP nimani avtomatlashtiradi, qo'lda yozilgan variant qachon foydali (maxsus siqish, gradient klipping), startap uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

16-variant. Bank: to'lov skoring modeli — eng arzon klaster konfiguratsiyasi (GPU-soat) qiyin

Soha: Moliya (bank)

Bankning ML platformasi 180 mln parametrli jadval-transformer skoring modelini 40 mln yozuvda 5 epoxa o'qitadi; byudjet 32 GPU'gacha, tarmoq variantlari 10, 25, 100 Gb/s, GPU-soat narxi tarmoqqa qarab farq qiladi. Sizdan s7 kalkulyatorini kengaytirib, (N, tarmoq, accum) kombinatsiyalari uchun qadam vaqti, E(N), epoxa davomiyligi va umumiy narxni hisoblash, eng arzon va eng tez variantlarni topish hamda natijani issiqlik xaritasida ko'rsatish talab qilinadi.

Model	TabTransformer 180 mln parametr, 6 GFLOP/namuna forward
Ma'lumot	40 mln yozuv, 5 epoxa
GPU	A100, 312 TFLOPS BF16, MFU 0.38; narx 2.1 $/GPU-soat
Tarmoq va qo'shimcha narx	10 Gb/s +0 %, 25 Gb/s +8 %, 100 Gb/s +20 %
N	1, 2, 4, 8, 16, 32
Accum	1, 2, 4, 8, 16
Paket/GPU	256
Cheklov	devor vaqti <= 12 soat

QADAMLAR

1. s7 estimate funksiyasini kengaytiring: qadam vaqti, 1 epoxa vaqti = (yozuvlar / (N*paket*accum)) * T_step, umumiy devor vaqti (5 epoxa), GPU-soat va narx (tarmoq ustamasi bilan).
2. N x tarmoq x accum (6 x 3 x 5 = 90 kombinatsiya) uchun jadval tuzing; devor vaqti 12 soatdan oshgan variantlarni chiqarib tashlang.
3. Eng arzon va eng tez kombinatsiyani toping; narx va vaqt bo'yicha Pareto chegarasini nuqtali grafikda chizing.
4. 10 Gb/s tarmoq uchun N x accum issiqlik xaritasi (rang — narx, $) va E(N) uchun ikkinchi xarita.
5. s3 formulasi bilan 180 mln parametr gradienti (BF16) uchun har tarmoqda all-reduce vaqtini hisoblang va kalkulyator T_comm bilan mos ekanini tekshiring; s6 Amdal bilan N=32 da nazariy chegarani solishtiring.
6. Sezgirlik tahlili: MFU 0.3 va 0.5, overlap 0.3 va 0.8 uchun eng arzon variant o'zgaradimi — jadval.
7. Xulosa (bank rahbariyati uchun 1 sahifa): tavsiya etilgan N, tarmoq, accum; nega katta accum bepul emas (yaqinlashuv, global paket); cheklovlar.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

17-variant. Genomika: variant patogenligi tasnifi — DDP va gradient to'plash kombinatsiyasi qiyin

Soha: Genomika (biotibbiyot)

Genomika laboratoriyasi 1024 o'lchamli ketma-ketlik belgilari bo'yicha mutatsiya patogenligini 4 sinfga ajratadi; 2 GPU'da xotira faqat 32 namunali mikro-paketga yetadi, lekin tajribalar 512 global paket talab qiladi. Sizdan haqiqiy DDP + no_sync bilan 2 ishchi x K mikro-paket sxemasini yozish, uning bitta 512 paket bilan gradient ekvivalentligini isbotlash, all-reduce sonini, vaqtni va yaqinlashuvni o'lchash hamda BatchNorm/SyncBatchNorm masalasini tahlil qilish talab qilinadi.

Ma'lumot	sintetik genomik belgilar 8192 x 1024 (4 sinf, siyrak motivlar), float32
Model	MLP 1024-512-128-4 (BatchNorm'li va BatchNorm'siz)
Mikro-paket (har rank)	32
Global paket	512 (2 ishchi x K=8)
K	1, 2, 4, 8
EPOCHS	2
LR qoidasi	0.02 * (global paket / 64)
Tekshiruv	max|g_512 - g_ddp_accum| < 1e-5

QADAMLAR

1. s1 DDP kodini genomik ma'lumotga moslang; BS=32 va K=1 baza vaqtini o'lchang.
2. DDP + no_sync bilan K=2, 4, 8 to'plashni yozing (oxirgi mikro-paketda sinxron); har K uchun vaqt, all-reduce soni, yakuniy yo'qotish va aniqlik jadvali; LR ni qoidaga ko'ra masshtablang.
3. Ekvivalentlik isboti: bir xil 512 namuna uchun (a) bitta jarayon 1x512 gradienti, (b) 2 ishchi x 8 x 32 DDP+accum gradienti (opt.step oldidan p.grad ni yig'ib oling) — max farq < 1e-5 ekanini ko'rsating.
4. Modelga BatchNorm1d qo'shing: ekvivalentlik buzilishini o'lchang; nn.SyncBatchNorm.convert_sync_batchnorm qo'llang va farq qanday o'zgarishini (to'plash ichida baribir qoladi) izohlang.
5. s3 formulasi bilan model gradienti hajmi (parametrlar sonini hisoblang) uchun 10 Gb/s da all-reduce vaqtini hisoblang; K bo'yicha epoxa uchun jami aloqa vaqtini jadvalga qo'shing.
6. s6 Amdal va s7 kalkulyatori bilan N=8 GPU, 10 Gb/s da K=1 va K=8 uchun E(8) ni baholang; o'lchangan 2 jarayon natijasi bilan taqqoslang.
7. Xulosa: 2 GPU xotira cheklovida 512 global paketga erishish rejasi, yaqinlashuv uchun LR/warmup eslatmasi, BatchNorm o'rniga LayerNorm tavsiyasi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

18-variant. 5G tarmoq: anomaliya detektori — straggler va asinxron SGD simulyatsiyasi qiyin

Soha: Telekom (5G infratuzilma)

5G operatori 64 ta bazaviy stansiya serverida tarmoq anomaliyalarini aniqlovchi modelni taqsimlangan tarzda o'qitadi; serverlar bir xil emas, ba'zilari 2-3 marta sekin. Sizdan sinxron SGD, zaxira ishchili sinxron SGD (eng sekin k tasini kutmaslik) va asinxron (eskirgan gradientli) SGD'ni NumPy'da simulyatsiya qilish, bir xil hisoblash byudjetida yaqinlashuv va devor vaqtini solishtirish hamda 2 jarayonli haqiqiy DDP'da straggler o'lchovini olish talab qilinadi.

Ma'lumot	sintetik KPI oqimi 20000 x 48 (2 sinf: normal/anomaliya, 5 % anomaliya)
Model (simulyatsiya)	logistik regressiya 48 og'irlik, NumPy'da qo'lda gradient
Ishchilar (simulyatsiya)	N = 16, qadam vaqti log-normal (sigma = 0.3 va 0.8)
Sxemalar	sinxron; sinxron + 2 zaxira (eng sekin 2 tasi kutilmaydi); asinxron (eskirish tau <= 4)
Haqiqiy DDP	2 jarayon, rank 1 kechikishi 0/2/5 ms
Byudjet	1600 gradient hisobi

QADAMLAR

1. KPI generatorini yozing va NumPy'da mini-paket gradientli logistik regressiyani amalga oshiring; 1 ishchi baza yaqinlashuv egri chizig'ini (yo'qotish vs gradient soni) oling.
2. Sinxron SGD simulyatsiyasi: har qadamda 16 ishchi vaqti log-normal taqsimotdan olinadi, qadam vaqti = maksimum; devor vaqti va yo'qotish egri chizig'i (sigma=0.3 va 0.8).
3. Zaxira ishchilar: har qadamda eng tez 14 ta gradient o'rtachalanadi; devor vaqti va yaqinlashuvni solishtiring.
4. Asinxron SGD: har ishchi o'z tezligida tugatganda eskirgan parametr bo'yicha hisoblangan gradient qo'llanadi (eskirish tau ni yozib boring); yo'qotish va o'rtacha tau jadvali; LR ni tau ga qarab kamaytirish ta'sirini sinang.
5. Haqiqiy o'lchov: s1 DDP kodini KPI ma'lumotiga moslang, rank 1 ga 0/2/5 ms sleep qo'shib E(2) jadvalini tuzing va simulyatsiya modeli E = T_qadam/(T_qadam + d) bilan solishtiring.
6. Uch sxema uchun 'bir xil devor vaqtida erishilgan yo'qotish' grafigini chizing (sigma=0.8 holatida) va s6 Amdal modeliga straggler ulushini qo'shib N=64 uchun E(64) ni baholang.
7. Xulosa: operator uchun qaysi sxema (yoki kombinatsiya) tavsiya etiladi, asinxron rejimning narxi (eskirgan gradient, yaqinlashuv kafolati), amaliy vositalar (torch elastic, Horovod) haqida izoh.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya

asosliligi — 20

19-variant. Aviatsiya: dvigatel telemetriyasi — ierarxik all-reduce va alfa-beta kechikish modeli qiyin

Soha: Aviatsiya

Aviakompaniya dvigatel telemetriyasidan qolgan resursni bashorat qiluvchi 95 mln parametrli modelni 4 tugun x 8 GPU klasterida o'qitadi: tugun ichida NVLink (600 GB/s), tugunlar orasida 10 yoki 100 Gb/s. Oddiy halqa 32 ta GPU'ni bitta halqaga tizadi va sekin bog'lamlar hammani tutib qoladi. Sizdan alfa-beta (kechikish + o'tkazuvchanlik) aloqa modelini yozish, yassi halqa va ierarxik (tugun ichida reduce-scatter/all-gather, tugunlar orasida halqa) sxemalarni NumPy'da simulyatsiya qilib vaqtini taqqoslash talab qilinadi.

Model	95 mln parametr (BF16 gradient = 190 MB), 14 GFLOP/namuna
Topologiya	4 tugun x 8 GPU; tugun ichida 600 GB/s, alfa 5 mks; tugunlar orasida 10 va 100 Gb/s, alfa 20 mks
Aloqa modeli	T = alfa * qadamlar + baytlar / BW (har bosqich uchun)
Sxemalar	yassi halqa (N=32); ierarxik 2 bosqich (8 ichida, 4 orasida)
Chelak hajmi	1, 25, 100 MB (qadamlar soniga ta'sir)
Simulyatsiya	L = 4096, 32 ishchi
GPU	A100, MFU 0.4, paket 64

QADAMLAR

1. s3 ring_allreduce asosida ierarxik all-reduce yozing: 1) har tugun ichida 8 ishchi reduce-scatter, 2) tugunlar orasida 4 ta bo'lak egasi halqa all-reduce, 3) tugun ichida all-gather; L=4096, 32 ishchi uchun natija to'g'riligini tekshiring va har ishchi yuborgan baytlarni bog'lam turi bo'yicha (ichki/tashqi) ajratib hisoblang.
2. Alfa-beta modelini amalga oshiring: har qadam vaqti = alfa + baytlar/BW; yassi halqa (31+31 qadam, eng sekin bog'lam tezligi bilan) va ierarxik sxema uchun 190 MB gradientda jami vaqtni 10 va 100 Gb/s uchun hisoblang.
3. Chelak hajmi 1, 25, 100 MB uchun all-reduce soni va alfa ulushini hisoblang; kichik chelakda kechikish (alfa) hukmron bo'lishini grafikda ko'rsating.
4. s7 kalkulyatoriga ierarxik T_comm ni qo'shing va N=32 uchun E(32) ni yassi va ierarxik sxemalarda, ikki tarmoqda solishtiring; overlap 0.5.
5. s1 DDP kodini telemetriya sintetik ma'lumotiga (6000 x 200, regressiya o'rniga 5 sinf) moslang va 2 jarayonda to'g'ri ishlashini ko'rsating; bucket_cap_mb=1 va 25 uchun vaqt farqini o'lchang (alfa ta'sirining CPU muqobili).
6. s6 Amdal bilan aloqa ulushidan N=32 va N=128 uchun chegarani baholang.
7. Xulosa: aviakompaniya klasteri uchun sxema, chelak hajmi va tarmoq tavsiyasi; NCCL ierarxik algoritmlarni qanday tanlashi haqida izoh.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun

tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

20-variant. E-hukumat: murojaatlarni tasniflash — o'qitish va inferens uchun to'liq masshtablash hisoboti qiyin

Soha: Davlat xizmatlari

Davlat xizmatlari portali kuniga 2 mln fuqaro murojaatini 24 ta idoraga yo'naltiradi; modelni 2 GPU'li serverda o'qitish va 8 yadroli serverda inferens qilish rejalashtirilgan, byudjet qat'iy. Sizdan laboratoriyaning barcha bosqichlarini bitta ssenariyga birlashtirish: DDP o'lchovi, masshtablash jadvali, gradient to'plash, parallel inferens profili, Amdal bahosi va kalkulyator — va yakuniy texnik hisobot (o'qitish SLA 4 soat, inferens p99 < 6 ms) tayyorlash talab qilinadi.

Ma'lumot	sintetik murojaat vektorlari 12000 x 768 (24 idora markazi + shovqin), 24 sinf
Model	MLP 768-512-256-24 (taxminan 532 ming parametr)
O'qitish	2 jarayon DDP, BS 64/128, EPOCHS 3, K = 1 va 4
Inferens	2 mln so'rov, SHARD 256/1024/4096, 1 va 2 ishchi, SLO p99 < 6 ms
Tarmoq (kalkulyator)	1, 10, 100 Gb/s
Real klaster bahosi	N = 2..16, ResNet-50 o'rniga shu model + BERT-base 110 mln
Hisobot	Markdown, 2 jadval + 3 grafik

QADAMLAR

1. s1 DDP: murojaat ma'lumotida BS=64 va 128, EPOCHS=3 uchun namuna/s, yo'qotish, aniqlik, parametr sinxronligi jadvali.
2. s2 masshtablash: HID=512 (ikki yashirin qatlam), GB=256 uchun 4 konfiguratsiya, S(2), E(2); kuchli rejim S(2) dan Amdal p ni toping.
3. s4 + DDP no_sync: K=4 to'plash bilan vaqt va all-reduce soni; 1x256 bilan gradient ekvivalentligini tekshiring.
4. s5 inferens: 2 mln so'rovni SHARD=256, 1024, 4096 da 1 va 2 ishchida; so'rov/s, p50/p99 jadvali; SLO p99 < 6 ms ni qondiruvchi eng katta SHARD; 8 yadro uchun S(8) ni Amdal bilan bashorat qiling va kunlik 2 mln so'rov necha daqiqa olishini hisoblang.
5. s3 + s7: 532 ming va 110 mln parametr uchun 1/10/100 Gb/s da all-reduce vaqti, N=2..16 da E(N) egri chiziqlari (kalkulyator) va ring simulyatsiyasi tekshiruvi.
6. Barcha natijalarni Markdown hisobotga yig'ing: o'qitish SLA 4 soat (12000 yozuvdan 2 mln ga ekstrapolyatsiya qilib, epoxa vaqtini namuna/s dan hisoblang) bajariladimi, inferens SLO bajariladimi, qaysi konfiguratsiya tavsiya etiladi.
7. Xulosa: ikki alohida tavsiya (o'qitish va inferens), xavflar (straggler, tarmoq, xotira) va keyingi qadamlar (profiler bilan tekshirish).

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20